CVE-2026-43284 / CVE-2026-43500 | CRITICAL | LPE

Dirty Frag

Detection Engineering Report

Wazuh 4.14.4 + auditd | Ubuntu - RHEL - Debian - Amazon Linux

WAZUH Author: Kislley Rodrigues | Wazuh Ambassador Program | May 2026
Github: https://github.com/mym0us3r/DIRTY-FRAG-Detection-with-Wazuh-4.14

Severity	**CRITICAL - Local Privilege Escalation (LPE) on Linux**
CVE IDs	CVE-2026-43284 (xfrm-ESP) / CVE-2026-43500 (RxRPC)
Affected	All Linux distributions since 2017
Exploit	Single compiled C binary - no race condition - no kernel offsets required
Fix	Commit f4c50a4034e6 | Mitigation: blacklist rxrpc / esp4 / esp6
Researcher	V4bel - https://github.com/V4bel/dirtyfrag
Wazuh Author	Kislley Rodrigues - Wazuh Ambassador Program
Date	May 2026

1. Executive Summary

CVE-2026-43284 / CVE-2026-43500 is a Linux kernel Local Privilege Escalation that chains two independent page-cache write primitives to achieve root from an unprivileged user account. Discovered and published by **V4bel**, the vulnerability exploits the same authencesn decrypt sink previously documented in Copy Fail (CVE-2026-31431), but through entirely different code paths that are not mitigated by the Copy Fail remediation.

A single compiled C binary achieves root on all major Linux distributions. No race condition. No heap spray. No kernel offsets required. The binary automatically selects between two exploit variants depending on the target environment.

The on-disk file remains unchanged - making traditional FIM completely blind, identical to the Copy Fail behavior.

CRITICAL: The Copy Fail mitigation (blacklisting algif_aead) does **NOT** protect against Dirty Frag. Both vulnerabilities share the same authencesn sink but are triggered through completely different code paths. Systems that applied Copy Fail remediation are still fully exposed to Dirty Frag.

Comparison with Similar Vulnerabilities

CVE	Dirty Cow (2016-5195)	Dirty Pipe (2022-0847)	Copy Fail (2026-31431)	Dirty Frag (2026-43284)
Race condition	Yes - multiple attempts	No	No - deterministic	No - deterministic
Can crash system	Yes	No	No	No
Portability	Limited	Version-specific	All distros 2017+	All distros 2017+
Exploit size	Kilobytes (C)	Hundreds of bytes	732 bytes Python stdlib	C binary - dual variant
FIM detection	Possible	Possible	Not possible	Not possible
Namespace required	No	No	No	No (RxRPC) / Yes (ESP)

Table 1 - Comparison with similar prior vulnerabilities

2. Technical Analysis

2.1 Root Cause

Dirty Frag exploits two independent kernel subsystems that share the same page-cache write sink. Both variants ultimately reach authencesn_decrypt(), which performs an in-place write into the page cache of the target file.

Variant	CVE	Trigger Path	Kernel Module
ESP	CVE-2026-43284	unshare(NEWUSER) -> XFRM SA -> vmsplice + splice -> esp_input()	esp4.ko / esp6.ko
RxRPC	CVE-2026-43500	add_key("rxrpc") -> socket(AF_RXRPC) -> vmsplice + splice -> rxkad_verify_packet_1()	rxrpc.ko

Table 2 - Dirty Frag variant comparison

2.2 Exploit Chain - Variant 1 (ESP / CVE-2026-43284)

Step	Syscall	Action	Detail
1	unshare(272)	Create user + net namespace	CLONE_NEWUSER|CLONE_NEWNET - gains CAP_NET_ADMIN inside new netns
2	netlink	Register XFRM SA	ESP SA with ESN flag - registers decrypt path
3	vmsplice(316)	Plant ESP header into pipe	24-byte crafted ESP header with attacker-controlled SPI
4	**splice(275)**	**Deliver /usr/bin/su page cache into pipe**	**Target file pages enter pipe buffer - TRIGGER**
5	splice(275)	Send pipe to UDP socket	esp_input() decrypt writes 4 bytes into page cache
6	execve(59)	Execute corrupted /usr/bin/su	Setuid binary runs embedded root shell ELF as UID 0

Table 3 - ESP exploit chain step by step

2.3 Exploit Chain - Variant 2 (RxRPC / CVE-2026-43500)

Step	Syscall	Action	Detail
1	add_key(248)	Plant rxkad session key K	K controls 8-byte STORE value via fcrypt_decrypt(C, K)
2	socket(41) AF_RXRPC	Open RxRPC socket	Triggers rxrpc.ko auto-load
3	vmsplice(316)	Plant RxRPC wire header into pipe	28-byte crafted header with pre-computed cksum
4	**splice(275)**	**Deliver /etc/passwd page cache into pipe**	**Target file pages enter pipe buffer - TRIGGER**
5	splice(275)	Send pipe to UDP socket	rxkad_verify_packet_1() writes 8 bytes into page cache
6	su -	Login as root	/etc/passwd root entry has empty password - PAM nullok accepts

Table 4 - RxRPC exploit chain step by step

2.4 Why Syscall Monitoring Extends FIM Coverage for This CVE

The write targets only the in-memory page cache. The kernel never marks the page as dirty, so the writeback mechanism never persists it to disk. Real-time FIM (inotify-based) operates at the VFS layer and monitors write events - since Dirty Frag bypasses the standard write path entirely, no inotify event is generated and real-time FIM correctly reports no change, because from the disk perspective there is none. Scheduled FIM would detect a hash divergence only if a scan runs during the brief window when the page cache is modified - a window measured in seconds against scan intervals measured in hours. After reboot, the page cache is flushed and the on-disk file is byte-for-byte identical to the original baseline. This is not a FIM limitation - it is a fundamental property of a page-cache-only write primitive. Behavioral detection via syscall monitoring is the complementary layer specifically designed to cover attack surfaces that do not produce file system events.

Traditional FIM approach:

read file from disk -> compute hash -> compare -> no anomaly reported

Dirty Frag reality:
on-disk /usr/bin/su = UNCHANGED page cache = CONTAINS ROOT SHELL ELF
on-disk /etc/passwd = UNCHANGED page cache = ROOT ENTRY HAS EMPTY PASSWORD

Real-time FIM (inotify) = BLIND - no VFS write event generated
Scheduled FIM = Unreliable - hours between scans, page cache flushed on reboot, disk always unchanged
Behavioral detection = ONLY reliable approach

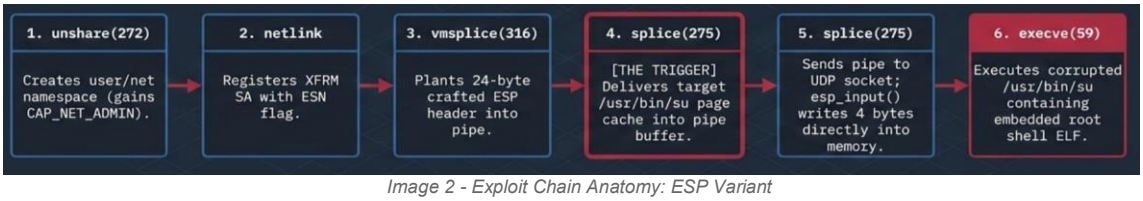

Image 2 - Exploit Chain Anatomy: ESP Variant

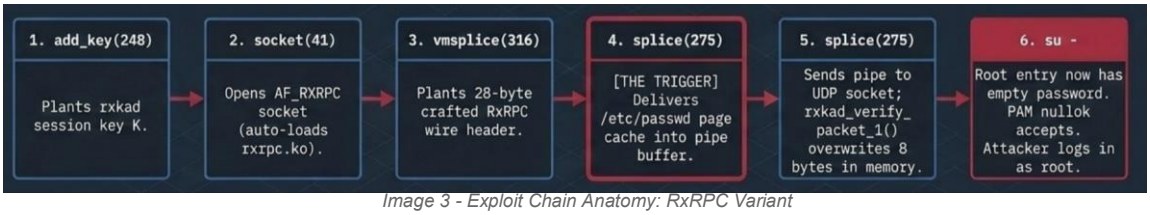

Image 3 - Exploit Chain Anatomy: RxRPC Variant

2.5 Attack Vector Breakdown: ESP vs. RxRPC

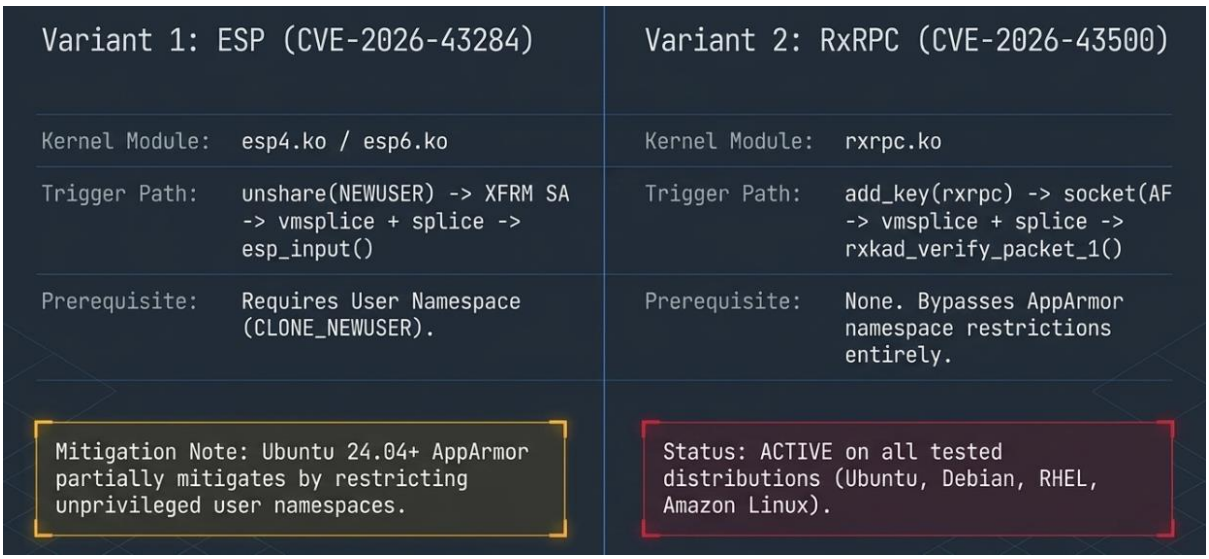

Image 1 – Attack Vector

Even with **FIM** scan intervals as low as **30 seconds**, detection is not guaranteed.

The exploitation window for Dirty Frag is **measured in seconds and the attacker obtains root before the next scan cycle completes**.

Any FIM alert for this CVE would be **post-exploitation** evidence, not prevention.

2.5 Confirmed Distributions and Kernels

Distribution	Kernel	Variant active	Reason	
Confirmed by @m0us3r - Ubuntu 24.04.2 LTS / Kernel 6.8.0-111-generic - RxRPC - AppArmor blocks ESP, rxrpc.ko loaded by default				
Confirmed by author (github.com/V4bel/dirtyfrag)				
Ubuntu 24.04.4	6.17.0-23-generic	RxRPC	Patched (f4c50a4034e6)	AppArmor blocks ESP + ESP patched in kernel 6.17
RHEL 10.1	6.12.0-124.49.1.el10_1.x86_64	ESP + RxRPC	Vulnerable	No namespace restriction, kernel predates ESP patch
openSUSE Tumbleweed	7.0.2-1-default	ESP + RxRPC	Vulnerable	No namespace restriction
CentOS Stream 10	6.12.0-224.el10.x86_64	ESP + RxRPC	Vulnerable	No namespace restriction
AlmaLinux 10	6.12.0-124.52.3.el10_1.x86_64	ESP + RxRPC	Vulnerable	No namespace restriction
Fedora 44	6.19.14-300.fc44.x86_64	ESP + RxRPC	Vulnerable	No namespace restriction

Table 5 - Distributions and kernels confirmed in lab

Ubuntu 24.04+ restricts CLONE_NEWUSER via AppArmor (***apparmor_restrict_unprivileged_userns=1***), which partially mitigates the ESP variant. The RxRPC variant bypasses this restriction entirely and remains active on all affected kernels.

3. Detection Architecture

The detection strategy is purely behavioral and does not depend on kernel version checks. Two independent layers provide complementary coverage.

3.1 Layer 1 - Behavioral Detection (auditd + Wazuh)

The auditd sensor captures syscalls specific to the exploit chain. The uid!=0 and auid>=1000 filters provide coverage beyond interactive users - including service accounts (www-data, postgres, jenkins), container processes, and web shells with no login session.

Engineering decision on uid vs auid: The ESP variant child process calls unshare(CLONE_NEWUSER) and maps itself as uid=0 inside the new user namespace. The Linux audit subsystem records the namespace-local uid (uid=0) in SYSCALL records. This means -F uid!=0 filters do NOT capture vmsplice and splice events from the exploit child. auid is set at login time and is immutable. The sensor rules for vmsplice and splice use -F auid>=1000 to correctly capture the real login user regardless of namespace state.

Auditd sensor rules (/etc/audit/rules.d/cve-dirty-frag.rules):

```
-a always,exit -F arch=b64 -S vmsplice -F auid>=1000 -F auid!=-1 -k dirty_frag_vmsplice
-a always,exit -F arch=b32 -S vmsplice -F auid>=1000 -F auid!=-1 -k dirty_frag_vmsplice
-a always,exit -F arch=b64 -S splice   -F auid>=1000 -F auid!=-1 -k dirty_frag_splice
-a always,exit -F arch=b32 -S splice   -F auid>=1000 -F auid!=-1 -k dirty_frag_splice
-a always,exit -F arch=b64 -S unshare  -F uid!=0                 -k dirty_frag_ns_escape
-a always,exit -F arch=b64 -S add_key  -F uid!=0                 -k dirty_frag_add_key
-a always,exit -F arch=b64 -S socket   -F a0=0x23 -F uid!=0      -k dirty_frag_rxrpc_socket
-w /usr/bin/kmod -p x                                           -k dirty_frag_modload
-a always,exit -F arch=b64 -S execve -F exe=/usr/bin/su -F auid>=1000 -F auid!=-1 -k dirty_frag_execve_su
```

3.2 Layer 2 - SCA Policy (cve-dirty-frag.yml)

Automated configuration checks every 12 hours via sca/cve-dirty-frag.yml. No kernel version required. Valid for Ubuntu, Debian, RHEL, Amazon Linux 2023, and SUSE.

3.3 Wazuh Rule Chain Architecture

Rule ID	Parent	Signal	Key	Depth	Level
80700	decoded_as=auditd	auditd anchor (Wazuh built-in)	-	0	0
200000	80700	vmsplice() - protocol header plant	dirty_frag_vmsplice	1	10
200001	80700	splice() - page cache plant	dirty_frag_splice	1	10
200002	80700	unshare(CLONE_NEWUSER) - ESP path	dirty_frag_ns_escape	1	12
200003	80700	add_key("rxrpc") - RxRPC key plant	dirty_frag_add_key	1	8
200004	80700	socket(AF_RXRPC) - RxRPC trigger	dirty_frag_rxrpc_socket	1	10
200005	80700	kmod/modprobe execution	dirty_frag_modload	1	12
200006	80700	execve /usr/bin/su - euid=root	dirty_frag_execve_su	1	6
200007	200001 + if_matched=200000	**CHAIN ESP: vmsplice + splice same pid/120s**	-	2	**15**
200008	200001 + if_matched=200004	**CHAIN RXRPC: AF_RXRPC + splice same pid/120s**	-	2	**15**

Table 6 - Wazuh rule chain architecture for CVE-2026-43284

200000	CVE-2026-43284 (Dirty Frag): vmsplice() called by non-root process [**audit.exe**] - protocol header plant into pipe	audit, linux, lpe, dirty_frag, cve_2026_43284	PCI_DSS HIPAA GDPR NIST_800_53 TSC MITRE	10	local_rules.xml	etc/rules
200001	CVE-2026-43284 (Dirty Frag): splice() called by non-root process [**audit.exe**] - target file page cache plant	audit, linux, lpe, dirty_frag, cve_2026_43284	PCI_DSS GDPR MITRE	10	local_rules.xml	etc/rules
200002	CVE-2026-43284 (Dirty Frag): unshare(CLONE_NEWUSER) by non-root [**audit.exe**] - ESP variant namespace privilege escalation path	audit, linux, lpe, dirty_frag, cve_2026_43284	PCI_DSS GDPR MITRE	12	local_rules.xml	etc/rules
200003	CVE-2026-43284 (Dirty Frag): add_key() called by non-root [**audit.exe**] - possible RxRPC session key plant for page cache write	audit, linux, lpe, dirty_frag, cve_2026_43284	PCI_DSS GDPR MITRE	8	local_rules.xml	etc/rules
200004	CVE-2026-43284 (Dirty Frag): socket(AF_RXRPC) opened by non-root [**audit.exe**] - RxRPC variant trigger path	audit, linux, lpe, dirty_frag, cve_2026_43284	PCI_DSS HIPAA GDPR NIST_800_53 TSC MITRE	10	local_rules.xml	etc/rules
200005	CVE-2026-43284 (Dirty Frag): kmod/modprobe execution - possible esp4/esp6/rxrpc module load [exe=**audit.exe** comm=**audit.command**]	audit, linux, lpe, dirty_frag, cve_2026_43284	MITRE	12	local_rules.xml	etc/rules
200006	CVE-2026-43284 (Dirty Frag): /usr/bin/su executed - review parent process and timing relative to vmsplice/splice events [**audit.exe**]	audit, linux, lpe, dirty_frag, cve_2026_43284	MITRE	6	local_rules.xml	etc/rules
200007	CVE-2026-43284 (Dirty Frag) EXPLOIT CHAIN DETECTED: vmsplice() + splice() from same process [pid=**audit.pid** exe=**audit.exe**] - page cache write attempt - IMMEDIATE INVESTIGATION REQUIRED	audit, linux, lpe, dirty_frag, cve_2026_43284	PCI_DSS HIPAA GDPR NIST_800_53 TSC MITRE	15	local_rules.xml	etc/rules
200008	CVE-2026-43284 (Dirty Frag) RXRPC EXPLOIT CHAIN DETECTED: socket(AF_RXRPC) + splice() from same process [pid=**audit.pid** exe=**audit.exe**] - RxRPC page cache write - IMMEDIATE INVESTIGATION REQUIRED	audit, linux, lpe, dirty_frag, cve_2026_43284	PCI_DSS HIPAA GDPR NIST_800_53 TSC MITRE	15	local_rules.xml	etc/rules

Figure 1 - Wazuh Rules Management: rules 200000-200008 deployed and active. Compliance tags (PCI_DSS, HIPAA, GDPR, NIST_800_53, MITRE ATT&CK) and severity levels confirmed. Rules 200007 and 200008 reach level 15 (Critical) on correlated chain detection.

4. Detection Rules

All rules are deployed in /var/ossec/etc/rules/local_rules.xml and validated with wazuh-analysisd -t (exit 0, zero warnings).

Each rule was individually confirmed with wazuh-logtest and validated in production via alerts.log and Wazuh Dashboard.

Rule	Level	Title	Description	Status
200000	10	vmsplice() non-root	vmsplice() syscall (316) by non-system user. Plants crafted protocol header into pipe. Unique to Dirty Frag. Filter: auid>=1000.	CONFIRMED
200001	10	splice() non-root	splice() syscall (275) by non-system user. Delivers target file page cache pages into pipe. Filter: auid>=1000.	CONFIRMED
200002	12	unshare(CLONE_NEWUSER)	unshare() by non-root. ESP variant prerequisite. On Ubuntu 24.04+, AppArmor logs AUDIT event even when blocked. Filter: uid!=0.	CONFIRMED
200003	8	add_key("rxrpc")	add_key() by non-root. RxRPC variant: plants session key K. Also auto-loads rxrpc.ko. Filter: uid!=0.	CONFIRMED
200004	10	socket(AF_RXRPC)	socket() with a0=0x23 (AF_RXRPC=35). RxRPC variant trigger. Legitimate use extremely rare outside AFS clients. Filter: uid!=0.	CONFIRMED
200005	12	kmod/modprobe	Execution of /usr/bin/kmod. Catches esp4/esp6/rxrpc module loads. No uid filter - covers kernel-triggered loads.	CONFIRMED
200006	6	execve /usr/bin/su	Execution of /usr/bin/su. euid=0 with auid=1000 confirms privilege escalation. Filter: auid>=1000 + exe=/usr/bin/su.	CONFIRMED
200007	**15**	**EXPLOIT CHAIN ESP**	Correlation: vmsplice() + splice() from same pid within 120s. Core pipe-plant sequence. Parent: 200001 + if_matched=200000.	CONFIRMED
200008	**15**	**EXPLOIT CHAIN RXRPC**	Correlation: socket(AF_RXRPC) + splice() from same pid within 120s. Parent: 200001 + if_matched=200004.	CONFIRMED

Table 7 - Detection rules summary for CVE-2026-43284

5. SCA Policy - cve-dirty-frag.yml

The SCA policy automates attack surface assessment across all Wazuh agents. No kernel version required. Valid for Ubuntu, Debian, RHEL, Amazon Linux 2023, and SUSE. Runs every 12 hours and on agent startup.

Check ID	Title	Condition	Risk
43284001	esp4 not loaded in memory	none: lsmod must NOT show esp4	**HIGH**
43284002	esp6 not loaded in memory	none: lsmod must NOT show esp6	**HIGH**
43284003	rxrpc not loaded in memory	none: lsmod must NOT show rxrpc	**HIGH**
43284004	esp4/esp6/rxrpc disabled via modprobe.d	any: conf file with install /bin/false	**HIGH**
43284005	auditd active and running	all: systemctl is-active returns 'active'	**HIGH**
43284006	CVE-2026-43284 sensor rules deployed	all: /etc/audit/rules.d/cve-dirty-frag.rules exists	**HIGH**

Table 8 - SCA policy checks for CVE-2026-43284

5.1 Validation Result - Lab Environment (agent wazuh5beta)

Check	Title	Result	Note
43284001	esp4 not loaded	**PASSED**	Module not present in memory
43284002	esp6 not loaded	**PASSED**	Module not present in memory
43284003	rxrpc not loaded	**PASSED**	Module not present in memory
43284004	Disabled via modprobe.d	**FAILED**	No blacklist config - modules can be auto-loaded
43284005	auditd active	**FAILED**	auditd not installed on baseline agent
43284006	Sensor rules deployed	**FAILED**	cve-dirty-frag.rules not deployed on baseline agent

Table 9 - SCA scan result in lab environment (Score: 50% - 3 passed / 3 failed)

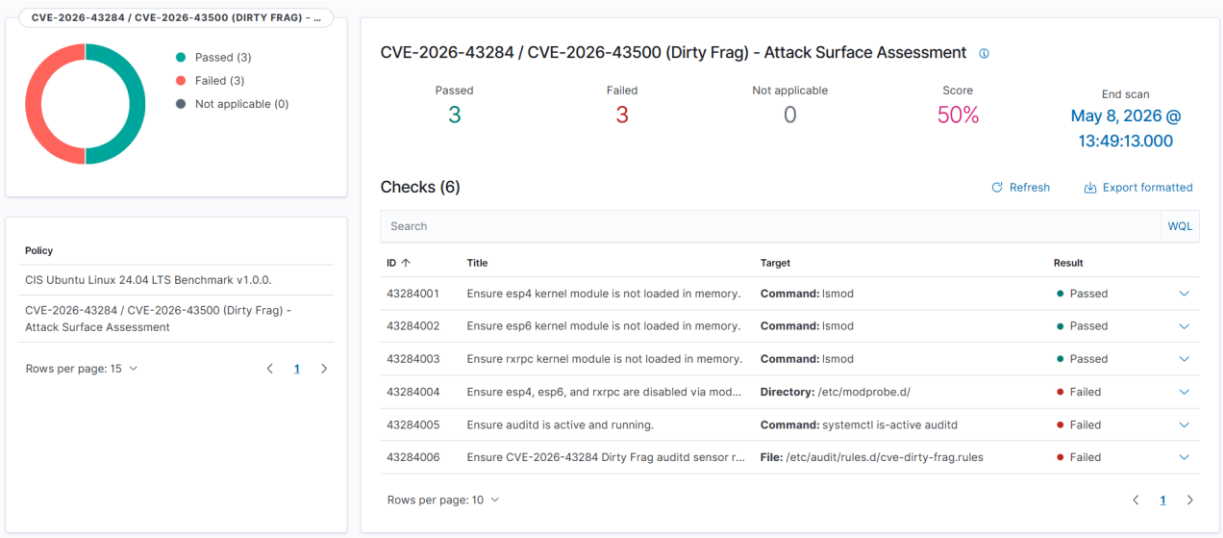

Figure 2 - SCA Policy: CVE-2026-43284 / CVE-2026-43500 Attack Surface Assessment. Score 50% (3 passed / 3 failed). Passed: esp4, esp6, rxrpc not loaded in memory. Failed: modprobe.d blacklist absent, auditd inactive, sensor rules not deployed - lab intentionally configured to simulate pre-mitigation state.

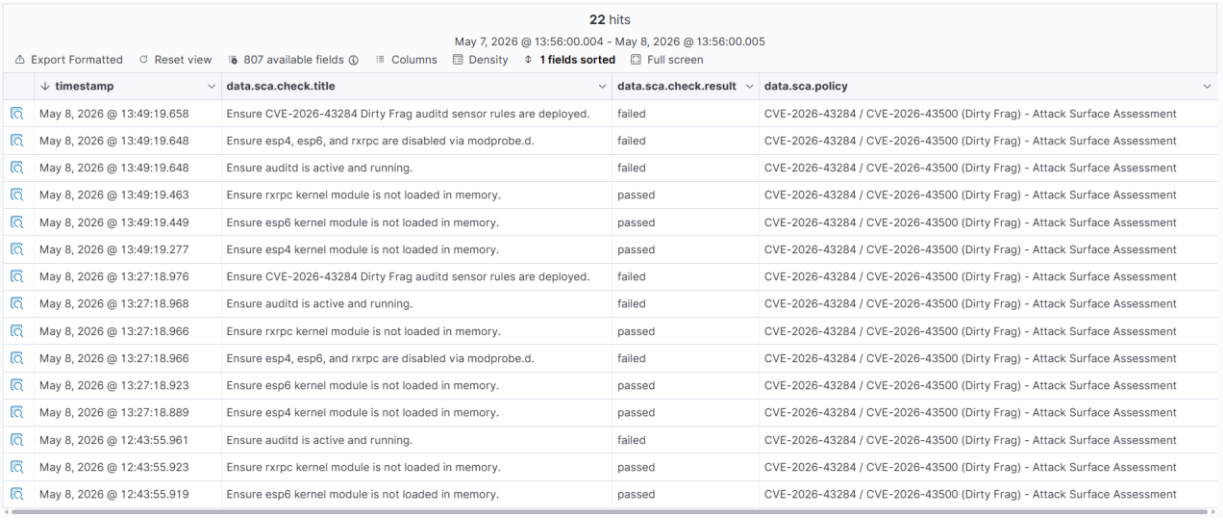

Figure 3 - Wazuh Discover: 22 SCA check results across 3 scan cycles on May 8, 2026 (12:43, 13:27 and 13:49 UTC). Persistent failed state on checks 43284004, 43284005, and 43284006 confirms continuous detection of unmitigated attack surface.

SCA Score: **50%** | 3 Passed | 3 Failed | 6 total checks. Score reaches 100% after full deployment of auditd + sensor rules + modprobe.d blacklist

6. Production Validation Evidence

All rules were validated in three stages: (1) wazuh-logtest with real auditd event samples for all 9 rules, (2) exploit execution on agent wazuh5beta (Ubuntu 24.04.2 LTS kernel 6.8.0-111-generic), and (3) confirmation in Wazuh Dashboard Discover.

6.1 Exploit Execution on Ubuntu 24.04.2 LTS (wazuh5beta)

The PoC was compiled and executed as user **kr (uid=1000)** on Ubuntu 24.04.2 LTS kernel 6.8.0-111-generic:

```
kr@wazuh5beta:~$ ./exp
root@wazuh5beta:~# date ; uname -a ; uptime ; id ; whoami
Sat May  9 04:59:08 AM UTC 2026
Linux wazuh5beta 6.8.0-111-generic #111-Ubuntu SMP PREEMPT_DYNAMIC Sat Apr 11 23:16:02 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
04:59:08 up 11:46,  5 users,  load average: 0.12, 0.24, 0.28
uid=0(root) gid=0(root) groups=0(root)
root
```

ROOT OBTAINED. User kr (uid=1000) with no privileges executed the PoC binary and obtained root shell (uid=0). Wazuh captured and alerted in real time.

6.2 Validation Evidence Table

Rule	Level	Test Performed	Result	Note
200000	10	wazuh-logtest: vmsplice() event auid=1000	FIRED	id: 200000 level: 10
200001	10	wazuh-logtest: splice() event auid=1000	FIRED	id: 200001 level: 10
200002	12	Real exploit: unshare() - AppArmor AUDIT event captured	FIRED	id: 200002 level: 12 - production
200003	8	wazuh-logtest: add_key() event uid=1000	FIRED	id: 200003 level: 8
200004	10	wazuh-logtest: socket(AF_RXRPC) event uid=1000	FIRED	id: 200004 level: 10
200005	12	Real exploit: kmod/modprobe execution captured	FIRED	id: 200005 level: 12 - production
200006	6	Real exploit: execve /usr/bin/su uid=kr euid=root	FIRED	id: 200006 level: 6 - production
200007	15	wazuh-logtest: vmsplice + splice same pid=55001	FIRED	id: 200007 level: 15
200008	15	wazuh-logtest: AF_RXRPC + splice same pid=55002	FIRED	id: 200008 level: 15

Table 10 - Production validation evidence for all detection rules

Figure 4 - Wazuh Discover: 52 alerts on May 8, 2026 - initial validation run. Rules 200005 (dirty_frag_modload) and 200006 (dirty_frag_execve_su) fired against the canonical PoC. Agent: wazuh5beta. MITRE ATT&CK T1068 - Exploitation for Privilege Escalation.

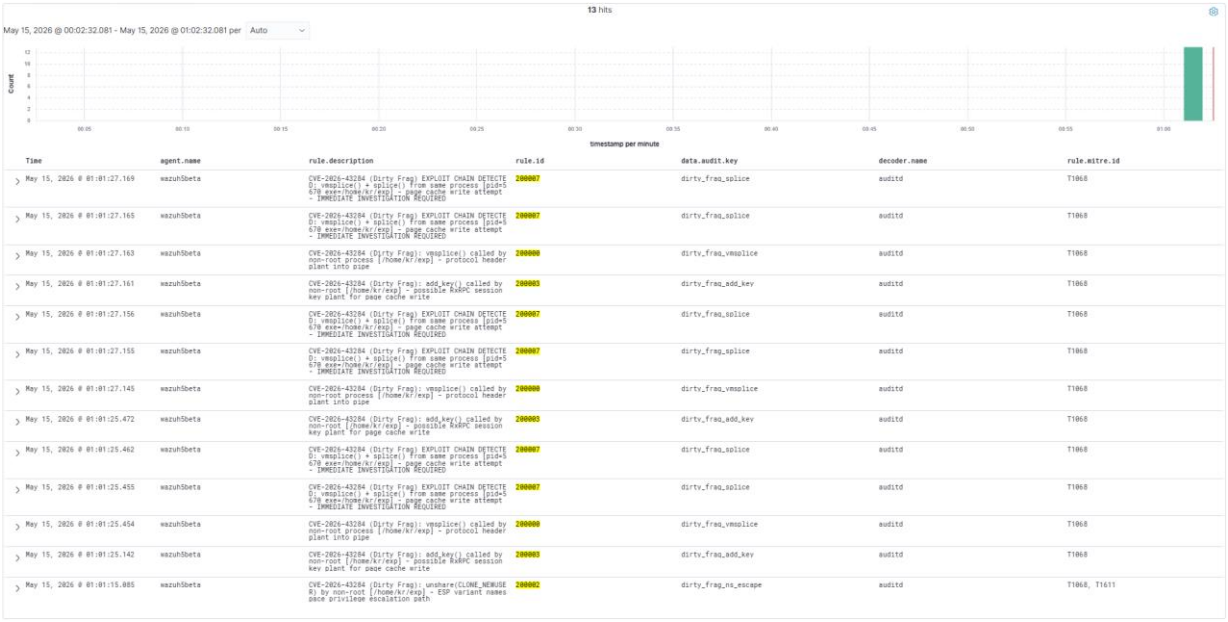

Figure 5 - Wazuh Discover: 13 alerts on May 15, 2026 - full validation run with auditd sensor rules deployed. Rule 200007 (EXPLOIT CHAIN DETECTED - level 15 Critical) fired on correlated vmsplice() + splice() from same process (pid=5670, exe=/home/kr/exp). Rules 200000, 200002, and 200003 confirmed active. IMMEDIATE INVESTIGATION REQUIRED.

wazuh-analysisd -t: exit 0 - zero warnings - 9 rules loaded - rules 200002/200005/200006 confirmed in Wazuh Dashboard Discover (agent=wazuh5beta, Today, 52 hits).

6.3 Lab Agents

ID	Name	IP	OS	Status
000	wazuh (manager)	192.168.1.6	Ubuntu 24.04.2 LTS	**Active**
003	wazuh5beta	DHCP	Ubuntu 24.04.2 LTS kernel 6.8.0-111	**Active**

Table 11 - Active agents in lab environment

7. Remediation Guide

7.1 Priority 1 - Apply the Kernel Patch

Update the distribution kernel package to a version that includes mainline commit f4c50a4034e6. All major distributions are shipping the fix.

7.2 Before Patching - Disable the Vulnerable Modules

RxRPC variant (CVE-2026-43500) - safe on all non-AFS systems:

```
echo 'install rxrpc /bin/false' >> /etc/modprobe.d/dirty-frag.conf
rmmod rxrpc 2>/dev/null || true
ESP variant (CVE-2026-43284) - only if no IPsec tunnels in use:
echo 'install esp4 /bin/false' >> /etc/modprobe.d/dirty-frag.conf
echo 'install esp6 /bin/false' >> /etc/modprobe.d/dirty-frag.conf
rmmod esp4 esp6 2>/dev/null || true
```

WARNING: Blacklisting esp4/esp6 breaks strongSwan/Libreswan IPsec tunnels. Do not apply on hosts running active IPsec connections. Blacklisting rxrpc does not affect IPsec, kTLS, SSH, or any general network stack.

7.3 Defense in Depth

Control	Action	Scope
Kernel patch	Apply commit f4c50a4034e6 via distribution kernel update	All distros
Module blacklist (rxrpc)	echo 'install rxrpc /bin/false' >> /etc/modprobe.d/dirty-frag.conf	Immediate - safe everywhere
Module blacklist (esp4/esp6)	Same pattern - only on non-IPsec hosts	Immediate - IPsec warning applies
auditd deployment	Install auditd + deploy cve-dirty-frag.rules + augenrules --load	Behavioral detection
Wazuh rules	Deploy local_rules.xml with rules 200000-200008	Alert generation
SCA policy	Deploy cve-dirty-frag.yml in /var/ossec/etc/shared/default/	Automated checks

Table 12 - Defense in depth controls for CVE-2026-43284

8. Relationship with Copy Fail (CVE-2026-31431)

	Copy Fail (CVE-2026-31431)	Dirty Frag (CVE-2026-43284/43500)
Sink	authencesn_decrypt seqno_lo write	authencesn_decrypt seqno_lo write
Trigger path	AF_ALG socket + splice via algif_aead	vmsplice + splice via esp_input or rxkad
Privileges required	None	None (RxRPC) / User namespace (ESP)
Mitigation overlap	Blacklist algif_aead	Blacklist rxrpc / esp4 / esp6
Cross-mitigation	algif_aead blacklist does NOT stop Dirty Frag	rxrpc blacklist does NOT stop Copy Fail
FIM detection	No	No
Auditd keys	copy_fail_*	dirty_frag_*
Wazuh rules	199600-199607	200000-200008

Table 13 - Comparison between Copy Fail and Dirty Frag

Both rulesets can coexist safely on the same Wazuh manager. The auditd key namespaces are separate and there are no rule ID conflicts.

9. Disclosure Timeline

Date	Event
2026-04-29	Copy Fail (CVE-2026-31431) public disclosure
2026-05-07	Dirty Frag public disclosure by V4bel - PoC published
2026-05-08	Detection lab completed - Wazuh 4.14.4 rules validated
2026-05-08	Community deliverable published

Table 14 - Disclosure timeline including Wazuh detection rules

As part of a commitment to open-source security, all lab artifacts have been released for public validation by Kislley Rodrigues - Wazuh Ambassador Program:

Repository	Link
DIRTY-FRAG Detection with Wazuh 4.14.4	https://github.com/mym0us3r/DIRTY-FRAG-Detection-with-Wazuh-4.14.4

10. Repository Structure

File	Description	Type
rules/local_rules.xml	9 Wazuh detection rules (200000-200008) with full engineering comments	Rules XML
auditd/cve-dirty-frag.rules	auditd sensor rules v2 - vmsplice, splice, unshare, add_key, AF_RXRPC, kmod, su	Sensor
sca/cve-dirty-frag.yml	SCA policy - 6 checks, kernel-version independent	SCA YAML
docs/	Wazuh Dashboard screenshots - production validation evidence	Screenshots
LICENSE	MIT License - unrestricted community use	MIT

Table 15 - Public repository structure

10.1 Repository - Wazuh Ambassador Program

Repository	Link
DIRTY-FRAG-Detection-with-Wazuh4.14.4	https://github.com/mym0us3r/DIRTY-FRAG-Detection-with-Wazuh-4.14.4

11. References

PoC - dirtyfrag (exp.c)
https://github.com/V4bel/dirtyfrag

Kernel Fix - f4c50a4034e6
https://github.com/torvalds/linux/commit/f4c50a4034e6

Mitigation Reference - CloudLinux
https://blog.cloudlinux.com/dirty-frag-mitigation-and-kernel-update

Related - Copy Fail (CVE-2026-31431)
https://github.com/mym0us3r/COPY-FAIL-Detection-with-Wazuh-4.14.4

MITRE ATT&CK T1068 - Exploitation for Privilege Escalation
https://attack.mitre.org/techniques/T1068/

Repository - DIRTY-FRAG Detection with Wazuh 4.14.4
https://github.com/mym0us3r/DIRTY-FRAG-Detection-with-Wazuh-4.14.4

Additional information about Wazuh and Ambassadors Program

wazuh.

Wazuh - Open Source XDR. Open Source SIEM

wazuh.
Ambassadors

Ambassadors Program | Wazuh

This report was produced as part of the Wazuh Ambassador Program. All detection rules, auditd sensor configuration and SCA policies were validated in a controlled lab environment running Wazuh 4.14.4 on Ubuntu 24.04.2 LTS (kernel 6.8.0-111-generic).

Detection Engineering: Kislley Rodrigues | Wazuh Ambassador | May 2026